Corporación Universitaria del Huila - Corhuila

Parcial Arquitectura de Software

Presentado por:

Carol Liceth Cruz Roa

Presentado a:

Luis Angel Vargas Narvaez

Facultad de Ingeniería

VI Semestre

Arquitectura De Software

Neiva - Huila

2026

1. Introducción

El presente documento tiene como objetivo realizar un diagnóstico arquitectónico del sistema ERP Iglesias, un sistema diseñado para la gestión administrativa de iglesias.

A partir del análisis del sistema se identificará el stack tecnológico utilizado, la estructura del proyecto y los principales módulos funcionales.

Posteriormente se propondrán mejoras arquitectónicas aplicando patrones de diseño creacionales y estructurales junto con principios SOLID con el fin de mejorar la mantenibilidad, escalabilidad y organización del código.

2. Contexto del Proyecto

ERP Iglesias es un sistema de planificación de recursos empresariales diseñado para apoyar la gestión administrativa de iglesias.

El sistema permite gestionar usuarios, personas, cursos, inscripciones, ofrendas y pagos, permitiendo centralizar la información administrativa y mejorar los procesos organizacionales.

El sistema está desarrollado con una arquitectura cliente-servidor donde el frontend consume servicios REST expuestos por el backend.

3. Diagnóstico Del Stack

El análisis del stack tecnológico permite entender cómo se comunican los diferentes componentes del sistema, cómo se gestionan los datos y qué herramientas se utilizan para el desarrollo, despliegue y ejecución de la aplicación.

A partir de la exploración del repositorio del proyecto y la ejecución local del sistema mediante contenedores Docker, fue posible identificar las principales tecnologías utilizadas en el frontend, backend, base de datos e infraestructura.

Este diagnóstico es fundamental para posteriormente proponer mejoras arquitectónicas mediante la aplicación de patrones de diseño creacionales, estructurales y principios SOLID que permitan mejorar la mantenibilidad y escalabilidad del sistema.

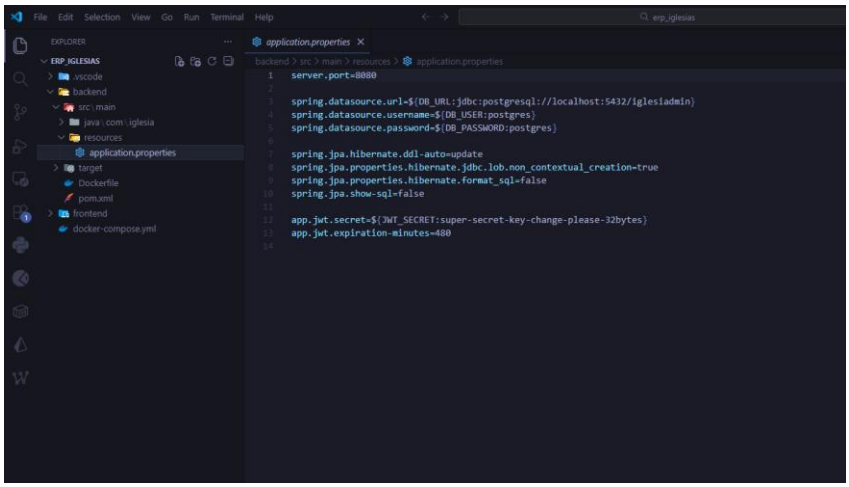
Backend

El backend del sistema está desarrollado utilizando Java con el framework Spring Boot, el cual permite construir aplicaciones robustas basadas en arquitectura en capas y servicios REST. Spring Boot facilita la creación de APIs y la gestión de dependencias mediante Maven, lo que permite organizar el proyecto de manera modular y escalable.

El backend es responsable de manejar la lógica de negocio del sistema, procesar las solicitudes provenientes del frontend y gestionar la comunicación con la base de datos.

El backend del sistema está desarrollado utilizando:

- Java
- Spring Boot
- Maven
- REST API



Frontend

El frontend del sistema está desarrollado utilizando el framework **Angular**, el cual permite construir aplicaciones web dinámicas basadas en componentes reutilizables. Angular utiliza **TypeScript** como lenguaje principal de desarrollo y Node.js como entorno para la gestión de dependencias y ejecución del proyecto durante el desarrollo.

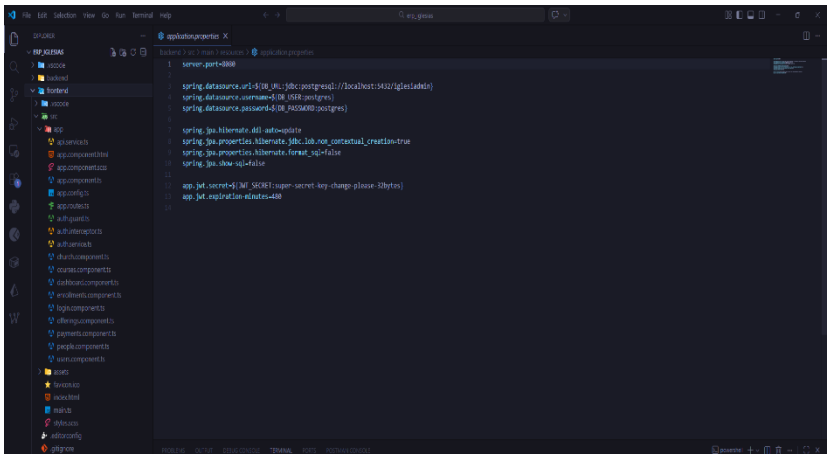
Además, el sistema utiliza **Nginx** como servidor web dentro del contenedor Docker para servir la aplicación en producción.

Durante el análisis del repositorio se identificó la siguiente estructura principal del **frontend:**

- Src
- angular.json
- package.json
- Dockerfile

Dentro de la carpeta **src** se encuentran los componentes principales de la aplicación Angular, organizados de la siguiente manera:

- src
  app
  components
  services
  guards
  interceptors
  routes
  config
  app.component.html
  app.component.scss
  app.component.ts



Base De Datos

El sistema ERP Iglesias utiliza **PostgreSQL 16** como sistema de gestión de base de datos relacional. PostgreSQL es una base de datos robusta y ampliamente utilizada en aplicaciones empresariales debido a su estabilidad, rendimiento y soporte para consultas complejas.

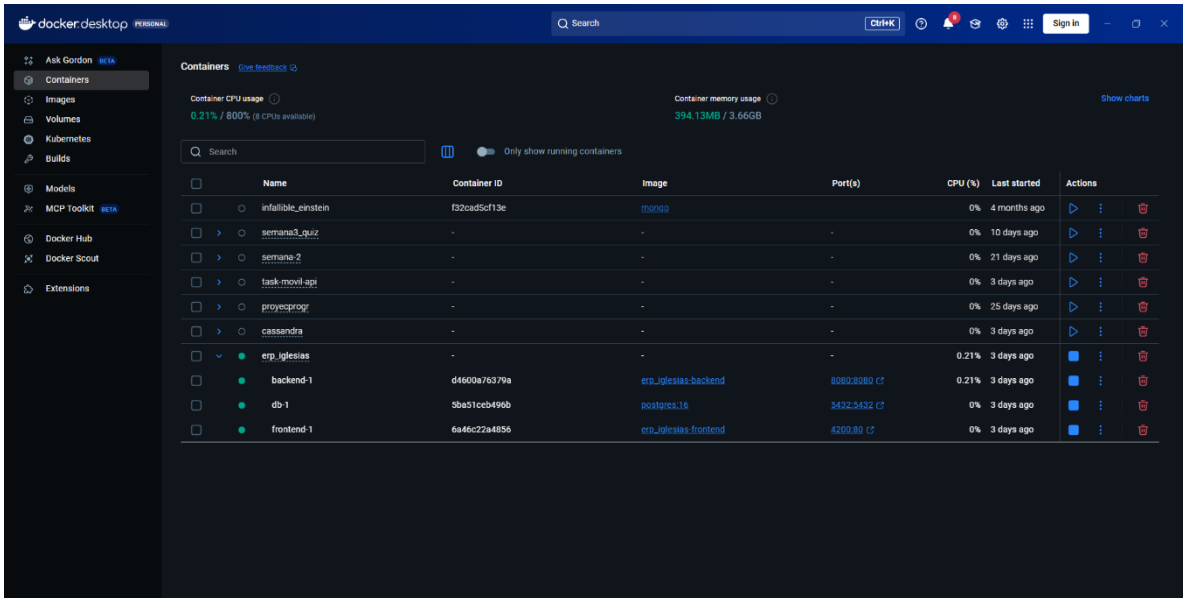
La base de datos es utilizada por el backend desarrollado en **Spring Boot** para almacenar y gestionar la información del sistema, incluyendo usuarios, personas, cursos, inscripciones, ofrendas y pagos.

Durante la ejecución del proyecto se identificó que la base de datos se ejecuta dentro de un contenedor Docker, lo que permite aislar el servicio y facilitar el despliegue del sistema en diferentes entornos.

El contenedor de base de datos utiliza el siguiente puerto para la conexión:

5432

Este puerto es el utilizado por PostgreSQL para permitir la comunicación con el backend del sistema.



4. Módulos del Sistema

Durante la exploración del sistema ERP Iglesias se identificaron varios módulos funcionales que permiten gestionar diferentes procesos administrativos dentro de la iglesia. Cada módulo representa una funcionalidad específica del sistema y permite organizar las operaciones de manera estructurada.

Los principales módulos identificados en el sistema son los siguientes:

Dashboard
Este módulo permite visualizar información general del sistema, como indicadores, resúmenes o estadísticas relevantes para la administración de la iglesia. Generalmente funciona como la pantalla principal del sistema.

Iglesia
Este módulo está orientado a la gestión de información relacionada con la iglesia, como datos institucionales o configuraciones generales del sistema.

Usuarios
Permite administrar los usuarios que tienen acceso al sistema. Dentro de este módulo se pueden crear, editar y gestionar las cuentas de usuario, así como controlar el acceso a las diferentes funcionalidades del sistema.

Personas
Este módulo permite gestionar la información de las personas registradas dentro de la iglesia, incluyendo datos personales y de contacto.

Cursos
Permite administrar los cursos o actividades formativas ofrecidas por la iglesia, incluyendo la creación y gestión de los mismos.

Inscripciones
Este módulo permite registrar la participación de las personas en los diferentes cursos o actividades disponibles dentro del sistema.

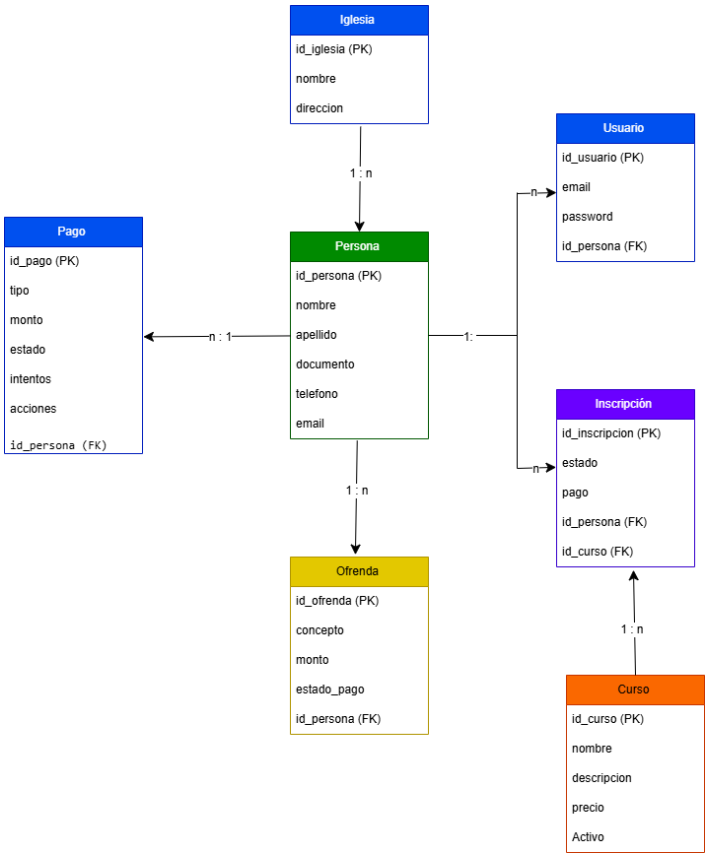
Ofrendas
Permite registrar las ofrendas o donaciones realizadas por los miembros de la iglesia, facilitando el control financiero de estas contribuciones.

Pagos
Este módulo está orientado al registro y gestión de pagos relacionados con diferentes actividades o servicios dentro del sistema.

5. Diagrama MER (Modelo Entidad Relación)

Link: Diagrama

https://drive.google.com/file/d/1_C01jbXZPhFfrfQenlTZuo7HjMLPQllR/view?usp=sharing



6. ADR – Propuesta (10 cambios)

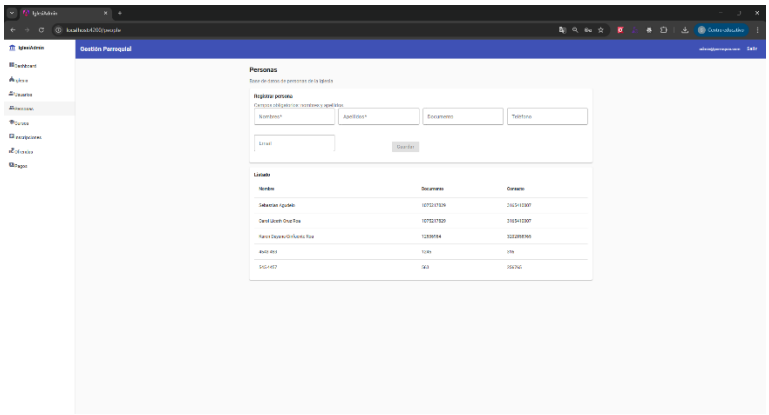
A partir del diagnóstico del sistema ERP Iglesias, del análisis de su estructura de código y de las pruebas funcionales realizadas sobre los diferentes módulos del sistema, se identificaron diversas oportunidades de mejora relacionadas con la validación de datos, la organización del código, el manejo de pagos y la separación de responsabilidades.

Con el fin de mejorar la mantenibilidad, escalabilidad y claridad arquitectónica del sistema, se proponen las siguientes decisiones arquitectónicas aplicando patrones de diseño creacionales y estructurales junto con principios SOLID.

ADR 1 – Validación de datos en el módulo Personas

Contexto
Durante las pruebas realizadas en el módulo **Personas** se detectó que el sistema permite registrar nombres utilizando valores numéricos.

Ejemplo de prueba realizada:
Nombre: 4543
Apellido: 453



El sistema guarda estos valores sin realizar validaciones.

Decisión
Agregar validaciones en el backend para garantizar que los campos nombre y apellido solo acepten texto.

Ejemplo de validación en Java:
@NotBlank
@Pattern(regexp = "^[a-zA-Z]+$")

Justificación
Esta mejora permite garantizar la calidad de los datos almacenados en la base de datos y evita inconsistencias en la información de las personas registradas.

Principio aplicado:
Fail Fast + Validación de dominio

ADR 2 – Validación del correo electrónico en el módulo Usuarios

En el módulo Usuarios el sistema permite registrar un usuario utilizando un correo electrónico y una contraseña.

Durante las pruebas se observó que el sistema muestra el mensaje “Usuario guardado”, pero no es claro si el correo electrónico tiene validación de formato.

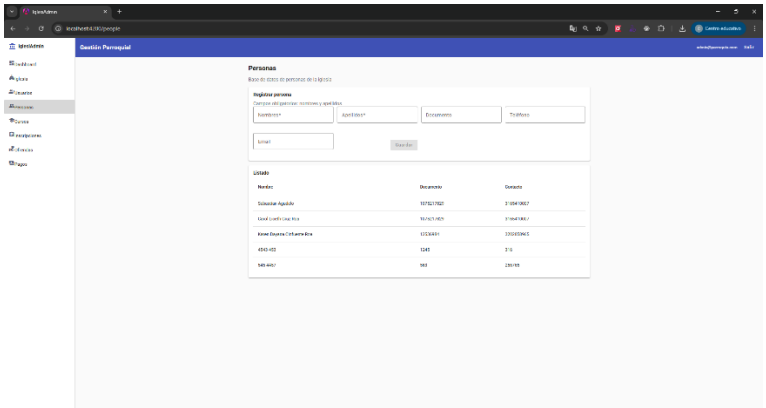
Contexto
Durante las pruebas realizadas en el módulo Personas se observó que el formulario de registro solicita los siguientes datos:

- Nombre
- Apellido
- Documento
- Teléfono
- Email

Sin embargo, al guardar la información y visualizar el listado de personas registradas, la tabla solo muestra:

- Nombre
- Documento
- Contacto

El campo correo electrónico no aparece en el listado, a pesar de haber sido registrado en el formulario.



Decisión
Agregar validaciones para asegurar que el campo correo electrónico tenga un formato válido.

Ejemplo:
@Email
@NotBlank

Justificación
Esto permite garantizar que los usuarios registrados en el sistema tengan correos electrónicos válidos y evita errores en futuras funcionalidades relacionadas con autenticación o comunicación.

Principio aplicado
Single Responsibility Principle (SRP)

ADR 3 – Registro de intentos en el proceso de pagos

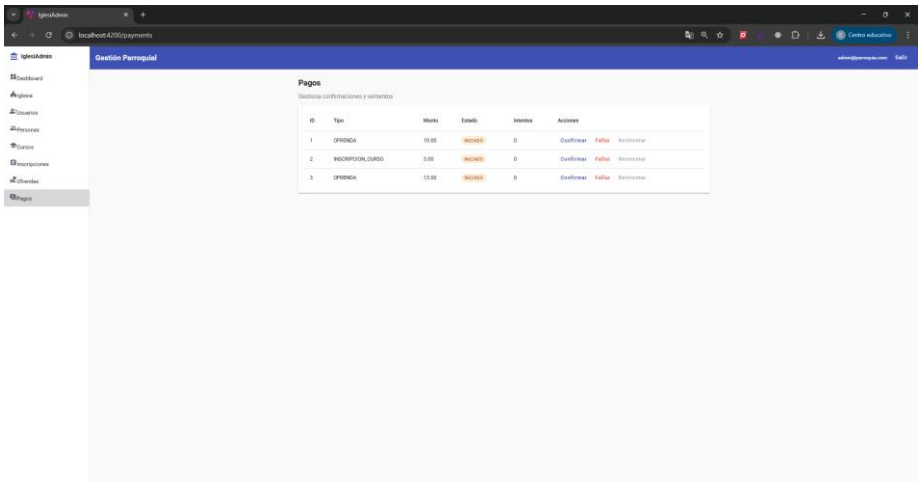
Contexto

Durante las pruebas realizadas en el módulo Pagos se observó que cada pago presenta los siguientes campos:

- ID
- Tipo
- Monto
- Estado
- Intentos
- Acciones (Confirmar, Fallar, Reintentar)

Además, cuando se crea un registro relacionado con pagos (por ejemplo una inscripción u ofrenda), el sistema genera automáticamente un pago con estado INICIADO.

Sin embargo, al presionar la opción Reintentar, no se observa un cambio en el campo intentos, por lo que no es posible identificar cuántas veces se ha intentado procesar un pago.



Decisión

Modificar la lógica del botón Reintentar para que cada vez que el usuario haga clic en esta acción, el sistema aumente el número de intentos del pago.

Por ejemplo:

- Intentos = 1 → primer intento de pago
- Intentos = 2 → segundo intento
- Intentos = 3 → tercer intento

De esta forma el sistema podrá registrar cuántas veces se ha intentado procesar un pago antes de confirmarlo o marcarlo como fallido.

Esto permitirá llevar un mejor control del proceso de pagos y facilitar el seguimiento de pagos que requieren múltiples intentos para completarse.

Además, permite aprovechar correctamente el campo intentos que ya existe en la tabla de pagos.

Principio aplicado
Single Responsibility Principle (SRP), mejorando la lógica de control del proceso de pagos.

ADR 4 – Validación del campo concepto en el módulo Ofrendas

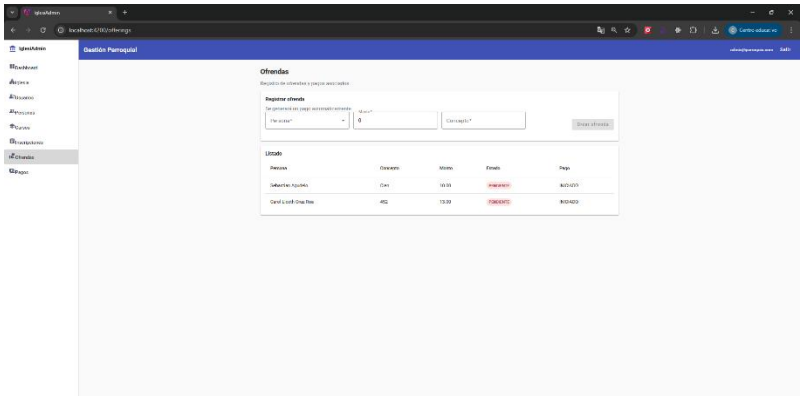
Durante las pruebas realizadas en el módulo **Ofrendas** se observó que el sistema permite registrar los siguientes datos:

- Persona
- Monto
- Concepto

Sin embargo, el campo **concepto** permite registrar valores numéricos o combinaciones que no representan una descripción clara de la ofrenda.
Ejemplo de prueba realizada: 452

Concepto

El sistema guarda este valor sin realizar validaciones.



Decisión
Agregar una validación para que el campo **concepto** solo permita **letras y texto descriptivo**, evitando el uso de números.

Ejemplo de valores correctos:

- Diezmo
- Ofrenda especial
- Donación

De esta manera se garantiza que el concepto describa correctamente el motivo de la ofrenda registrada.

Esta mejora permitirá mejorar la calidad de la información financiera registrada en el sistema y evitar registros incorrectos dentro del módulo de ofrendas.

Principio aplicado
Validación de dominio, asegurando la integridad de los datos almacenados en el sistema.

ADR 5 – Control de estados en el proceso de pagos

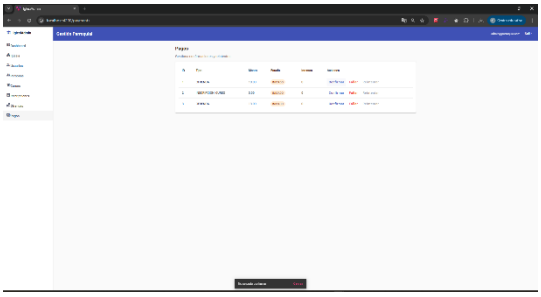
Durante las pruebas realizadas en el módulo **Pagos** se observó que cada pago presenta los siguientes campos:

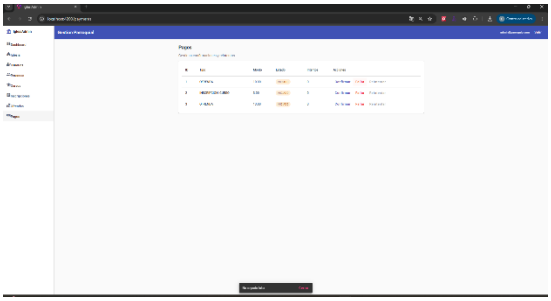
• ID
• Tipo
• Monto
• Estado
• Intentos
• Acciones (Confirmar, Fallar, Reintentar)

Actualmente el sistema permite ejecutar las acciones **Confirmar** y **Fallar**, pero en algunas ocasiones aparece un mensaje indicando que **no se pudo confirmar** o **no se pudo fallar**, lo que genera confusión sobre el estado actual del pago.

Además, se identificó que el campo **Estado** debe controlar correctamente el flujo del proceso de pago para evitar acciones que no correspondan.

Por ejemplo, cuando se crea un pago, este inicia con estado **PENDIENTE** o **INICIADO**, y desde ese estado es donde deben ejecutarse las acciones disponibles. Sin embargo, si el pago ya se encuentra **Confirmado** o **Fallido**, el sistema no debería permitir ejecutar nuevamente estas acciones.





Decisión

Definir claramente los **estados del pago** dentro del sistema para que el proceso sea más claro y controlado.

Los estados del pago serán los siguientes:

Estado	Descripción
Pendiente	El pago fue creado pero aún no ha sido validado por el sistema.
Confirmado	El pago fue verificado correctamente y aceptado.
Fallido	El pago no se pudo procesar o fue rechazado.

El flujo de funcionamiento será el siguiente:

• Cuando se crea un pago → el estado inicial será **Pendiente**.
• Si el pago se procesa correctamente → el sistema ejecuta **Confirmar** y el estado cambia a **Confirmado**.
• Si ocurre un error en el proceso → el sistema ejecuta **Fallar** y el estado cambia a **Fallido**.

Una vez que el pago se encuentre en estado **Confirmado** o **Fallido**, no se permitirán más cambios sobre ese registro.
Esto permite mantener un control claro del proceso de pagos y evita que el sistema ejecute acciones que no corresponden al estado actual del pago.

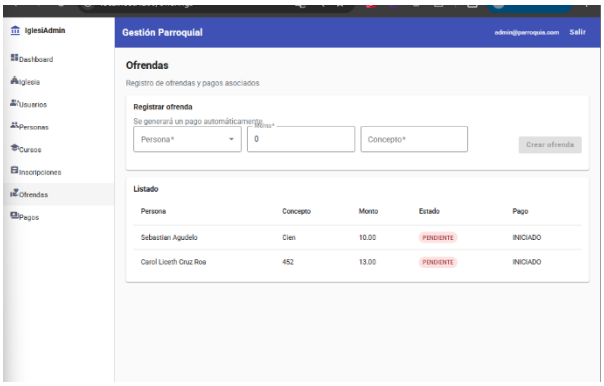
Principio aplicado
Single Responsibility Principle (SRP), ya que el campo **Estado** se encarga exclusivamente de controlar la situación actual del pago dentro del sistema.
.

ADR 6 – Validación del campo monto en el módulo Ofrendas
Contexto

Durante las pruebas realizadas en el módulo **Ofrendas** se observó que el sistema permite registrar los siguientes datos:

• Persona
• Monto
• Concepto

Al realizar pruebas ingresando valores en el campo **Monto**, el sistema permite registrar valores sin validar si el monto corresponde a un número válido o positivo. Esto puede generar registros incorrectos dentro del sistema financiero de la aplicación.



Decisión
Agregar validaciones en el backend para garantizar que el campo **Monto** solo permita valores numéricos positivos.

Ejemplo de validación en Java:
@NotNull
@Positive
Esto asegurará que los valores registrados representen correctamente una cantidad de dinero válida.

Justificación
Implementar esta validación permitirá mejorar la integridad de los datos financieros registrados en el sistema y evitar inconsistencias en los registros de ofrendas.

Principio aplicado
Validación de dominio, asegurando que los datos cumplan las reglas del negocio antes de almacenarse.

ADR 7 – Implementar patrón Repository para acceso a datos

El sistema utiliza Spring Boot con acceso a base de datos PostgreSQL.

Decisión
Centralizar el acceso a datos utilizando el patrón **Repository**, permitiendo separar la lógica de persistencia de la lógica de negocio.
Ejemplo:
PersonRepository
CourseRepository
PaymentRepository

Justificación
Mejora la organización del código y facilita la mantenibilidad.

Patrón aplicado
Repository Pattern

ADR 8 – Implementación del patrón DTO para transferencia de datos

El sistema ERP Iglesias utiliza un frontend desarrollado en Angular que consume servicios REST del backend desarrollado en Spring Boot.
Actualmente es posible que las entidades de base de datos se envíen directamente al frontend.

Decisión

Implementar **DTO (Data Transfer Object)** para controlar los datos que se envían desde el backend hacia el frontend.

Ejemplos de DTO:

- PersonDTO
- CourseDTO
- PaymentDTO

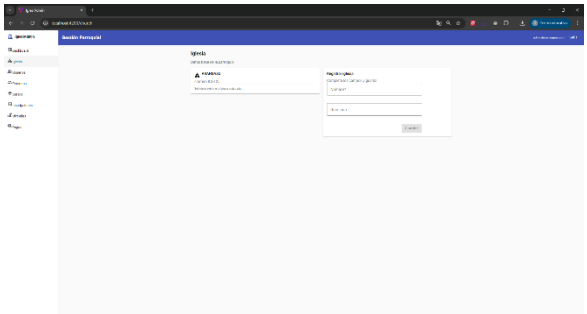
Justificación
El uso de DTO permite desacoplar las entidades de base de datos de la API pública del sistema, mejorar la seguridad y facilitar futuras modificaciones en el modelo de datos sin afectar el frontend.

Patrón aplicado
DTO Pattern

ADR 9 – Soporte para múltiples iglesias en el sistema

Durante las pruebas realizadas en el módulo Iglesia se observó que el sistema permite registrar una iglesia con los siguientes datos:
• Nombre
• Dirección
Al guardar la información, el sistema muestra el siguiente mensaje:
“Solo debe existir una iglesia registrada”.

Además, al intentar registrar una segunda iglesia el sistema no permite hacerlo, lo que indica que existe una restricción que limita el sistema a una única iglesia.



Decisión
Modificar la lógica actual del sistema para permitir el registro de múltiples iglesias, en caso de que el sistema sea utilizado por diferentes sedes o congregaciones.
Esto implicaría que las entidades relacionadas (personas, cursos, ofrendas, pagos e inscripciones) puedan asociarse a una iglesia específica dentro del sistema.

Permitir múltiples iglesias haría que el sistema sea más flexible y escalable, ya que podría ser utilizado por diferentes sedes o comunidades religiosas sin necesidad de instalar una nueva instancia del sistema para cada iglesia.
Esta mejora permitiría adaptar el sistema a escenarios más amplios de uso.

Principio aplicado
Open/Closed Principle (OCP), permitiendo extender la funcionalidad del sistema sin afectar el funcionamiento actual.

ADR 10 – Control de activación de cursos en el sistema

Durante las pruebas realizadas en el módulo Cursos se observó que el sistema permite registrar los siguientes datos:

• Nombre
• Descripción
• Precio

Al guardar un curso, el sistema muestra en el listado los siguientes campos:

• Nombre
• Precio
• Activo (Sí)

Esto indica que el sistema maneja un estado de activación para los cursos. Sin embargo, en la interfaz no se observa una opción clara para activar o desactivar un curso una vez que ha sido creado.
Esto puede generar dificultades cuando un curso ya no se ofrece, pero sigue apareciendo disponible en el sistema.

Decisión

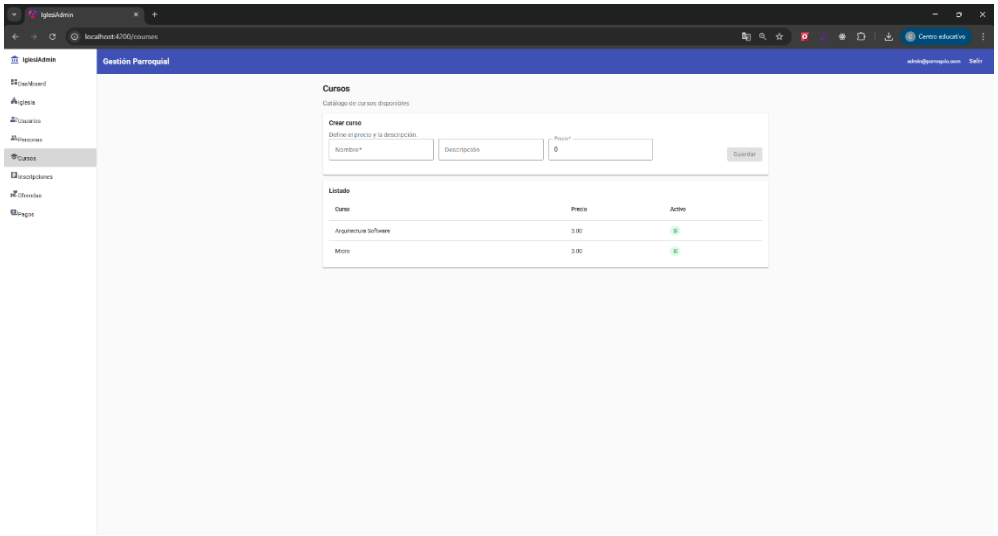
Agregar una funcionalidad que permita cambiar el estado de los cursos entre activo e inactivo desde la interfaz del sistema.
De esta forma, los cursos que ya no estén disponibles podrán marcarse como inactivos sin necesidad de eliminarlos de la base de datos.

Esta mejora permitirá administrar mejor los cursos registrados en el sistema y evitar que cursos antiguos o no disponibles aparezcan como activos para nuevas inscripciones.

Además, mantener los cursos inactivos en lugar de eliminarlos permite conservar el historial de información relacionada con inscripciones y pagos.

Principio aplicado

Single Responsibility Principle (SRP), ya que el estado del curso se encarga exclusivamente de controlar su disponibilidad dentro del sistema.



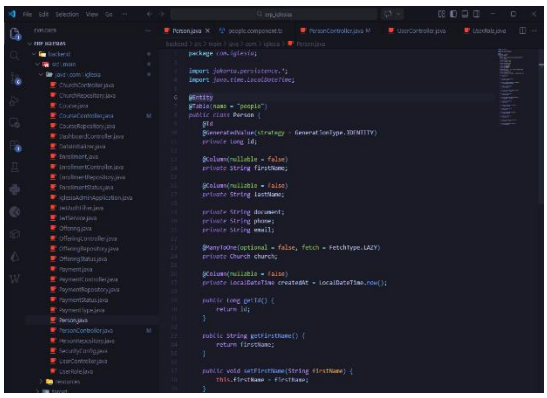
7. Implementación (5 cambios)

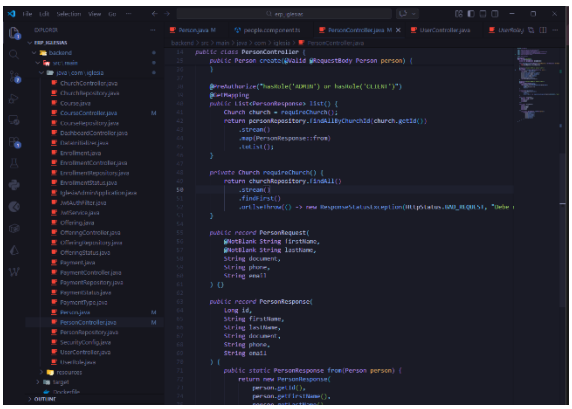
ADR 1 – Validación de datos en el módulo Personas

Para mejorar la calidad de los datos ingresados al sistema se implementaron validaciones en los campos de nombre y apellido dentro de la entidad Person.

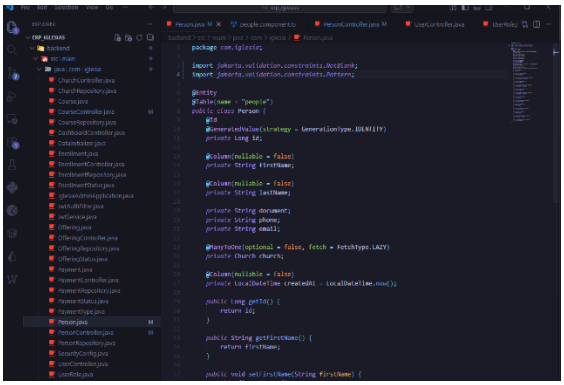
Con esta implementación el sistema evita registrar nombres vacíos o con valores numéricos, mejorando la integridad de los datos almacenados en la base de datos.

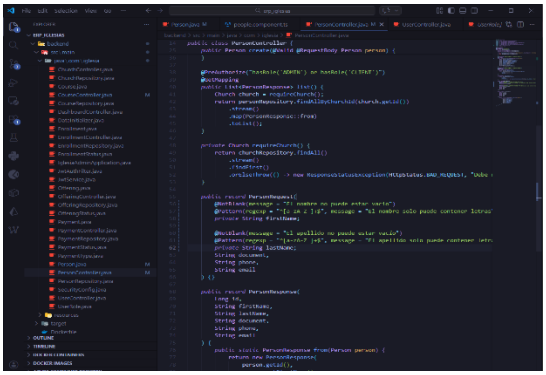
Como Estaba



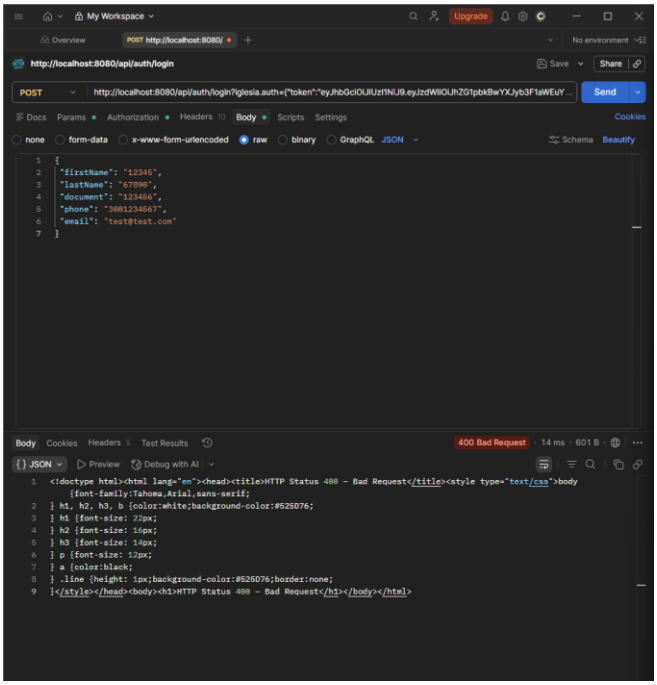


Después





Prueba- Postman



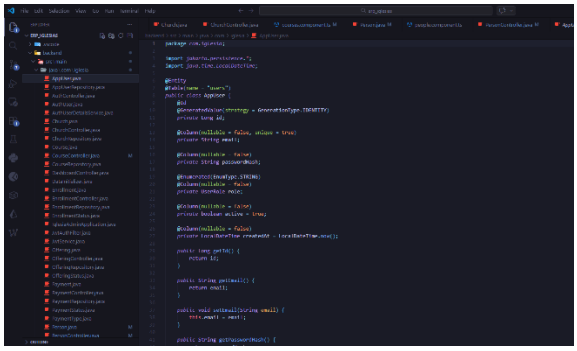
ADR 2 – Validación del correo electrónico en el módulo Usuarios

El módulo Usuarios permite registrar un usuario con correo y contraseña, pero al guardar no había validación de formato. Esto podía generar errores en futuras funcionalidades (autenticación, comunicación).

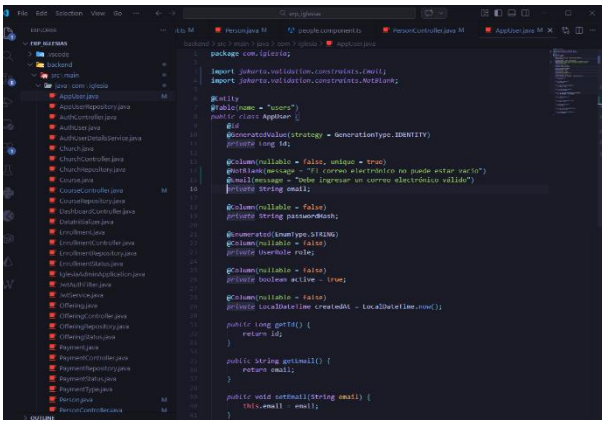
Agregar validaciones @NotBlank y @Email al campo de correo electrónico en appUser.java.

Se asegura que los usuarios tengan correos válidos y se evitan errores futuros.

Como Estaba



Después



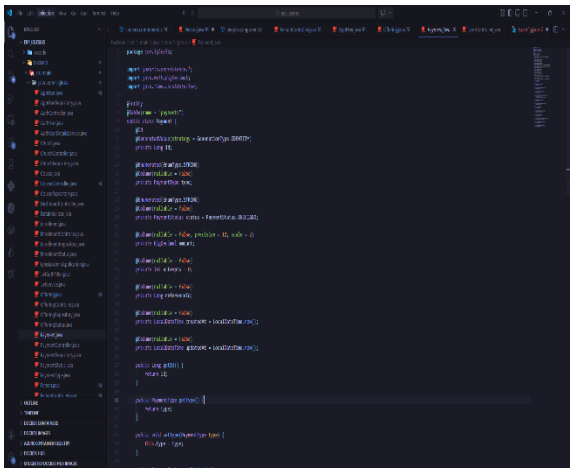
ADR 3 – Registro de intentos en el proceso de pagos

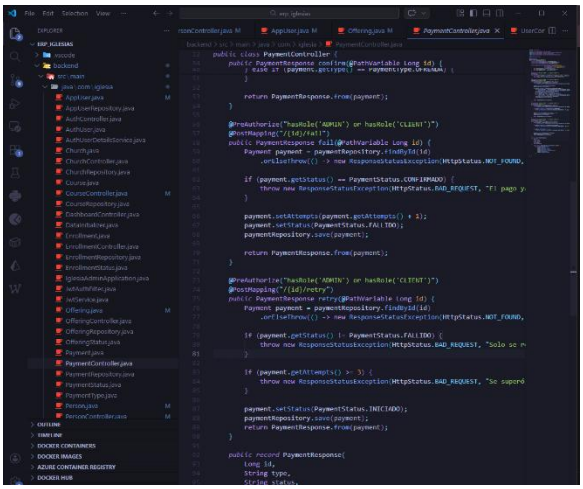
Se modificó la acción de Reintentar en PaymentController.java para que cada vez que se vuelva a procesar un pago fallido, el campo attempts se incremente en 1. Esto permite llevar un control claro de cuántas veces se ha intentado procesar un pago, manteniendo restricciones como máximo 3 reintentos y solo pagos con estado FALLIDO.

Beneficios:

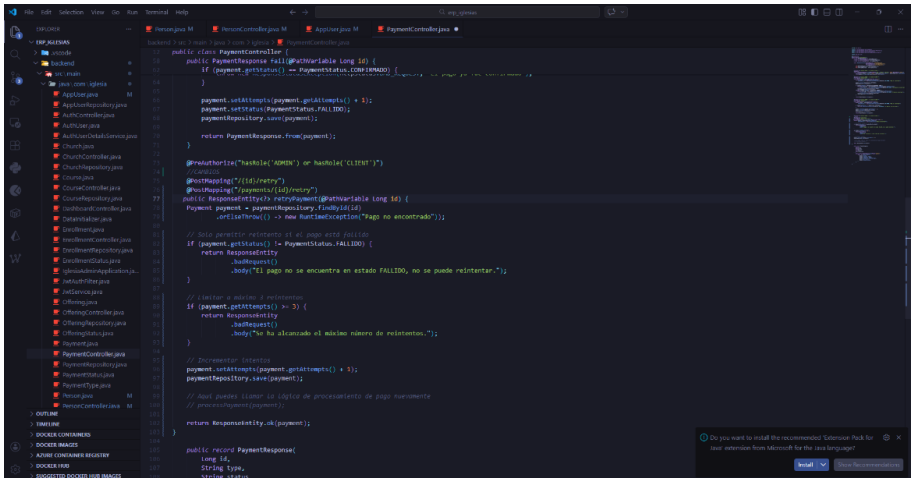
- Mejora la trazabilidad y control del proceso de pagos.
- Asegura la integridad de la información, evitando confusión sobre el número de intentos.
- Es buena práctica aplicar Single Responsibility Principle (SRP), ya que el campo attempts se encarga exclusivamente de registrar los intentos del pago.

Como Estaba





Después



ADR 4 – Validación del campo concepto en el módulo Ofrendas

En esta mejora se implementó la validación del campo concept en la entidad Offering.java para garantizar que las ofrendas registradas en el sistema contengan descripciones claras y consistentes. Específicamente, se agregaron las siguientes anotaciones de validación de Jakarta Bean Validation:

- @NotBlank (message = "El concepto no puede estar vacío"): asegura que el campo no quede vacío al registrar una ofrenda.
- @Pattern(regexp = "^[a-zA-Z]+$", message = "El concepto solo puede contener letras y espacios"): limita el contenido del campo únicamente a letras y espacios, evitando que se registren números o símbolos.

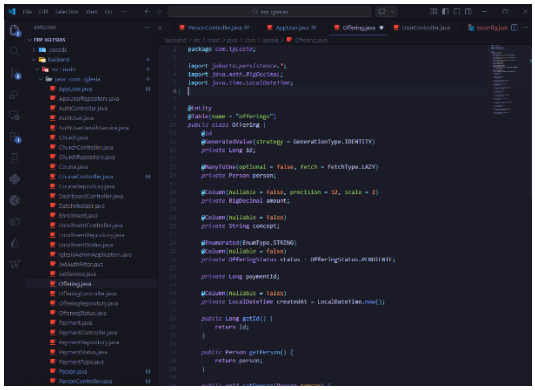
Objetivo de la implementación:

- Evitar registros incorrectos o confusos en el módulo de ofrendas.
- Garantizar la calidad de la información financiera almacenada.
- Facilitar la interpretación y el uso de los datos en reportes y análisis posteriores.

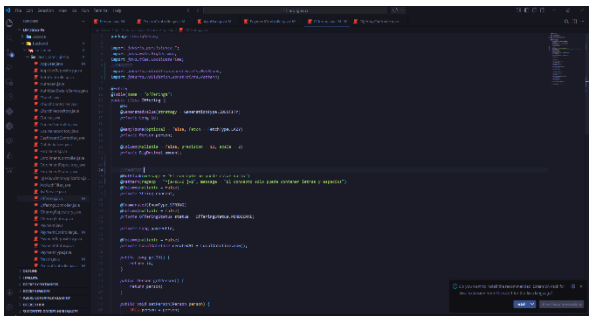
Por qué es buena práctica:

- Aplica la validación de dominio, asegurando que cada entidad cumpla con las reglas del negocio antes de persistir los datos en la base de datos.

Como Estaba



Después



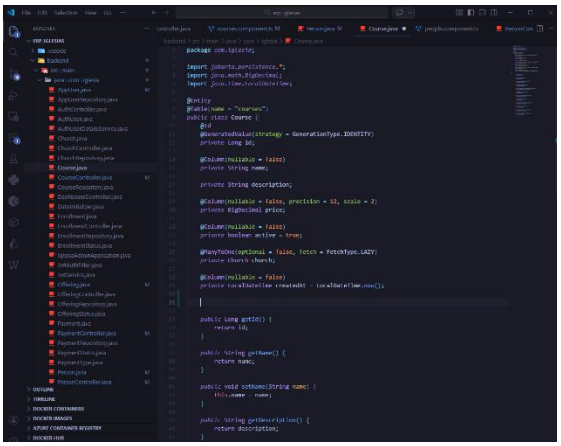
ADR 10 – Control de activación de cursos en el sistema

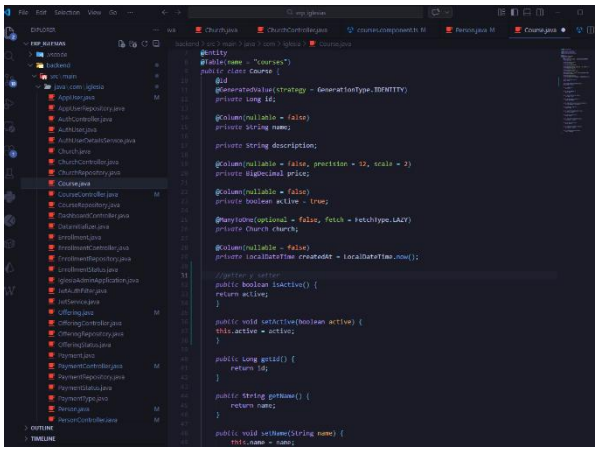
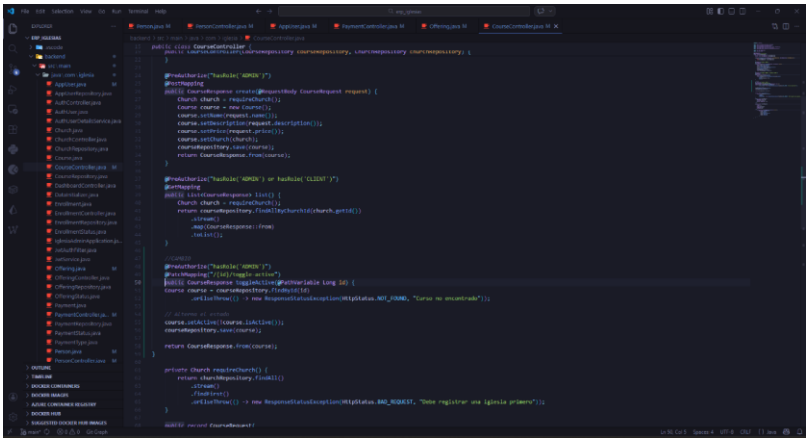
Se implementó un método toggleActive() en CourseController.java que permite activar o desactivar cursos existentes, alternando el valor del campo active. Esto facilita gestionar la disponibilidad de los cursos sin eliminarlos de la base de datos.

Beneficios:

- Permite controlar qué cursos están disponibles para inscripción.
- Mantiene el historial de cursos y sus registros asociados.
- Es buena práctica aplicar Single Responsibility Principle (SRP), ya que el campo active se encarga exclusivamente de controlar la disponibilidad del curso.

Como Estaba





Después